

Building, Training, and Analyzing Monolingual Transformer Language Models in Hindi and Nepali

Raj k jain

Roll No.: 2025201036

Email: raj.jain@students.iiit.ac.in

International Institute of Information Technology, Hyderabad

September 11, 2026

Abstract

This report provides a comprehensive scientific account of building, training, analyzing, and finetuning two completely independent decoder-only Transformer language models from first principles in two Indian languages: Hindi (Model H, higher-resource) and Nepali (Model L, lower-resource). The project encompasses the entire lifecycle of language model development without utilizing any pretrained models, tokenizers, or third-party Transformer abstractions.

In **Phase 1**, independent monolingual corpora exceeding 600M tokens were collected, cleaned, deduplicated, and split, followed by training custom SentencePiece BPE tokenizers from scratch (32,000 vocab for Hindi; 16,000 vocab for Nepali). In **Phase 2**, custom 6-layer, 8-head, 512-dimension decoder-only Transformers featuring Rotary Position Embeddings (RoPE), pre-norm RMSNorm, SwiGLU feed-forward blocks, and student-implemented causal masking were pretrained from scratch using AdamW and cosine scheduling with mixed precision and resumable checkpointing. Full evaluation across intrinsic metrics (PPL, BPB), open-ended generation (BLEU-4, chrF, ROUGE-L, Distinct-1/2, repetition rate), and multi-head attention dynamics was conducted. In **Phase 3**, deterministic synthetic comparative- and transitive-reasoning datasets with rigorous leakage controls (20% held-out entity names, test-only attribute) were generated in native phrasing, and models were finetuned using masked-prompt loss, demonstrating substantial accuracy gains (Hindi: 12.0% $\rightarrow$ 47.75%; Nepali: 0.5% $\rightarrow$ 24.5%) and attention sink reorganization. Finally, an **Optional Bonus Ablation** was executed by retraining the 36.84M parameter Hindi model with positional embeddings completely removed, proving empirically and theoretically how self-attention degenerates into word-order permutation invariance, catastrophic autoregressive loops (repetition surges to 71.6%–88.5%), and a 47-fold explosion in perplexity.

Contents

1	Introduction and Project Overview	3
1.1	Objectives	3
1.2	Project Constraints and Compliance	3
2	Phase 1: Corpus Collection and Tokenizer Construction	5
2.1	Language Selection Justification	5
2.2	Dataset Collection and Preprocessing Pipeline	5
2.3	Tokenizer Training and Fertility Analysis	6
3	Phase 2: Model Implementation, Pretraining, and Evaluation	7
3.1	Architectural Formulation	7
3.1.1	Input Representation and Rotary Position Embeddings (RoPE)	7
3.1.2	Multi-Head Attention and Causal Masking	7
3.1.3	Transformer Block: Normalization and SwiGLU FFN	7
3.1.4	Weight Tying and Parameter Accounting	8
3.2	Pretraining Setup and Optimization	8
3.3	Language Modeling and Generation Evaluation Results	8
3.4	Phase 2 Attention Analysis	9
4	Phase 3: Reasoning Finetuning, Attention Analysis, and Cross-Phase Synthesis	10
4.1	Synthetic Reasoning Dataset Construction	10
4.1.1	Task Specifications	10
4.1.2	Leakage Prevention Controls	10
4.2	Supervised Finetuning Protocol	10
4.3	Pretrained vs. Finetuned Reasoning Results	11
4.4	Post-Finetuning Attention Redistribution	11
4.5	Synthesis: Addressing the Four Core Research Questions	12
5	Bonus: Transformer Ablation Study — No Positional Embeddings	13
5.1	Theoretical Rationale: Attention Without Position	13
5.2	Ablated Implementation and Empirical Invariance Test	13
5.3	Comparative Evaluation Results	13
5.4	What Breaks Without Position Information?	14
6	Conclusion and Deliverable Index	16
6.1	Key Scientific Conclusions	16
6.2	Master Deliverable and Artifact Index	17

1 Introduction and Project Overview

Modern autoregressive language modeling predominantly utilizes the decoder-only Transformer architecture. While large multilingual models have become standard in industrial NLP, training monolingual models for non-English languages from first principles provides critical insights into tokenization fertility, linguistic structure, resource constraints, and attention specialization.

1.1 Objectives

The primary objective of this project is to build, pretrain, evaluate, and analyze two completely disjoint, independent language models:

1. **Model H (Higher-Resource):** Hindi, written in the Devanagari script, benefiting from extensive public corpora and high ecosystem support.
2. **Model L (Lower-Resource):** Nepali, also written in Devanagari, but characterized by distinct morphological structures, vocabulary distributions, and lower digital text availability.

1.2 Project Constraints and Compliance

In accordance with course guidelines:

- **No Pretrained Models or Tokenizers:** No HuggingFace Transformer model classes, pretrained tokenizers, or pretrained weights were permitted or used.
- **Zero Cross-Language Leakage:** Model H and Model L share zero training data, tokenizers, vocabularies, or weights.
- **From-Scratch Implementation:** Token embeddings, Rotary Position Embeddings (RoPE), multi-head causal self-attention, SwiGLU feed-forward networks, and causal masking were implemented directly using primitive PyTorch layers (`nn.Linear`, `nn.Embedding`, `nn.Parameter`).
- **Resumability:** Checkpoints save model weights, optimizer state, scheduler state, current step, and configuration.

Table 1: System Specifications and Hyperparameters at a Glance

Attribute	Model H (Hindi)	Model L (Nepali)
Language Tier	Higher-Resource Indian Language	Lower-Resource Allowed Indian Language
Script	Devanagari	Devanagari
Vocabulary Size	32,000 tokens	16,000 tokens
Context Length (T)	512 tokens	512 tokens
Architecture	Decoder-only Transformer	Decoder-only Transformer
Layers (N) / Heads (h)	6 layers / 8 heads	6 layers / 8 heads
Model Dim (d_{model}) / d_k	512 / 64	512 / 64
FFN Dim (d_{ff})	2,048 (SwiGLU)	2,048 (SwiGLU)
Positional Embedding	Rotary Position Embeddings (RoPE)	Rotary Position Embeddings (RoPE)
Layer Normalization	Pre-norm RMSNorm ($\epsilon = 10^{-6}$)	Pre-norm RMSNorm ($\epsilon = 10^{-6}$)
Weight Tying	Input & Output Embeddings Tied	Input & Output Embeddings Tied
Total Trainable Parameters	36,837,888	28,645,888
Training Compute	NVIDIA RTX 3050 Laptop GPU	NVIDIA RTX 3050 Laptop GPU

2 Phase 1: Corpus Collection and Tokenizer Construction

2.1 Language Selection Justification

Hindi (Model H): Hindi is spoken by over 600 million people worldwide and is one of the most widely used languages in the Indian subcontinent. It has substantial digital presence, with curated repositories such as the AI4Bharat Sangraha corpus providing hundreds of millions of clean tokens, enabling the target of $\sim 500\text{M}$ tokens to be achieved.

Nepali (Model L): Nepali is an officially recognized Indian language (prominently spoken in Sikkim and northern West Bengal) and belongs to the permitted lower-resource language tier. Although Nepali shares the Devanagari script with Hindi, its vocabulary, agglutinative inflectional morphology, verbal conjugation system, and case-marking conventions differ sharply.

2.2 Dataset Collection and Preprocessing Pipeline

The dataset creation pipeline was engineered for high memory safety and checkpoint-based resumption. Data was ingested into structured SQLite databases (`hindi_state.db` and `nepali_state.db`) with document deduplication hashes.

1. **Public Corpus Streaming:** Extracted from verified splits of the AI4Bharat Sangraha corpus via streaming chunks to prevent RAM exhaustion.
2. **Manual Data Collection:** Collected from scanned literature and public domain regional books in PDF format. Pages were processed using Optical Character Recognition (OCR) via Tesseract (`tesseract-ocr-hin` and `tesseract-ocr-nep`) followed by rule-based cleanup.
3. **Text Normalization and Filtering:** Unicode normalization (NFC/NFKC), removal of broken codepoint sequences, Devanagari character verification, duplicate document filtering via MD5 hashing, and removal of non-linguistic noise.
4. **Splits:** Deterministically partitioned into 90% train, 5% validation, and 5% test sets.

Table 2: Phase 1 Corpus Statistics and Token Distribution

Dataset Metric	Hindi (Model H)	Nepali (Model L)
Total Documents	1,323,072	1,836,292
Total Tokens (All Splits)	681,200,436	775,992,967
Training Tokens	613,059,508	698,466,193
Validation Tokens	34,070,464	38,763,387
Test Tokens	34,070,464	38,763,387
Manual Training Tokens	6,777,503 (1.11%)	322,028 (0.05%)
Downloaded Training Tokens	606,282,005 (98.89%)	698,144,165 (99.95%)
Database Size on Disk	8.0 GB	11.0 GB

Discussion on Manual Data Target: The project specification targeted $\geq 20\%$ manual data. However, due to the massive volume of verified Sangraha data required to exceed 600M tokens, manual OCR collection accounted for 1.11% in Hindi and 0.05% in Nepali. This shortfall is explicitly reported and treated as a critical ecological factor when interpreting results.

2.3 Tokenizer Training and Fertility Analysis

Independent SentencePiece Byte-Pair Encoding (BPE) tokenizers were trained from scratch on 10 million sample lines per language.

- **Model H Vocabulary (32,000 tokens):** Hindi’s rich literature and larger data volume allow a 32k vocabulary, minimizing token fragmentation.
- **Model L Vocabulary (16,000 tokens):** Nepali’s smaller corpus was allocated a 16k vocabulary to avoid subword sparsity and unobserved long-tail tokens.

Table 3: Tokenizer Quality and Subword Diagnostics

Metric	Hindi Tokenizer	Nepali Tokenizer
Algorithm	SentencePiece BPE	SentencePiece BPE
Vocabulary Size	32,000	16,000
Average Characters per Token	4.12	4.59
Fertility Rate (Tokens / Word)	1.24	1.40
Validation UNK Rate	0.0485%	0.1585%

Key Observation: Nepali exhibits noticeably higher fertility (1.40 vs. 1.24 tokens/word) and a $3.27\times$ higher out-of-vocabulary UNK rate (0.1585% vs. 0.0485%). This structural fragmentation creates greater cognitive load for Model L, as words are split into smaller grammatical particles.

3 Phase 2: Model Implementation, Pretraining, and Evaluation

3.1 Architectural Formulation

The model is a decoder-only autoregressive Transformer language model constructed without high-level library abstractions.

3.1.1 Input Representation and Rotary Position Embeddings (RoPE)

Token IDs are mapped to dense embeddings $X \in \mathbb{R}^{B \times T \times d_{\text{model}}}$. Rather than standard absolute sinusoidal embeddings, Rotary Position Embeddings (RoPE) were implemented. RoPE incorporates relative position directly into the self-attention dot product by rotating the query and key vectors in two-dimensional coordinate slices:

$$R_{\Theta,t}^{d_k} = \text{diag}\left(R_{\theta_1,t}, R_{\theta_2,t}, \dots, R_{\theta_{d_k/2},t}\right), \quad \text{where } R_{\theta_i,t} = \begin{pmatrix} \cos(t\theta_i) & -\sin(t\theta_i) \\ \sin(t\theta_i) & \cos(t\theta_i) \end{pmatrix} \quad (1)$$

with base $\theta_i = 10000^{-2(i-1)/d_k}$. When queries and keys are rotated, their inner product satisfies:

$$(R_t q)^T (R_s k) = q^T R_{s-t} k \quad (2)$$

This formulation preserves relative positional distance invariant under absolute translation while restricting the maximum sequence length to the precomputed rotary frequencies ($T = 512$).

3.1.2 Multi-Head Attention and Causal Masking

For input $H \in \mathbb{R}^{B \times T \times d}$, linear projections compute queries, keys, and values:

$$Q = HW_Q, \quad K = HW_K, \quad V = HW_V \quad (3)$$

Tensors are reshaped to (B, h, T, d_k) . The causal attention weight matrix is computed as:

$$A = \text{softmax}\left(\frac{R(Q)R(K)^T}{\sqrt{d_k}} + M\right), \quad M_{ij} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases} \quad (4)$$

The factor $1/\sqrt{d_k}$ counteracts variance growth in large dot products that would push the softmax into vanishing gradient regimes.

Empirical Causal Mask Verification: To prove that token t cannot observe future tokens $t + 1$, Sequence A was fed to the model and logits at position $t = 2$ were recorded. Token $t + 3$ was then altered in Sequence B. The difference in logits at $t = 2$ was measured:

$$\max |\text{Logits}_A[2, :] - \text{Logits}_B[2, :]| = \mathbf{0.000000} \quad (5)$$

while position $t = 4$ exhibited a difference of 6.7968. This proves that future information leakage is strictly prevented.

3.1.3 Transformer Block: Normalization and SwiGLU FFN

Each of the 6 layers utilizes Pre-Norm Root Mean Square Normalization (RMSNorm):

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}} \odot \gamma \quad (6)$$

Pre-norm maintains a clean residual highway from input to output, preventing gradient attenuation in deep stacks. The Feed-Forward Network employs the SwiGLU activation function:

$$\text{FFN}_{\text{SwiGLU}}(x) = (\text{SiLU}(xW_1) \otimes xW_3) W_2 \quad (7)$$

where $W_1, W_3 \in \mathbb{R}^{d \times d_{\text{ff}}}$ and $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d}$ with $d_{\text{ff}} = \frac{2}{3}(4d) = 2048$.

3.1.4 Weight Tying and Parameter Accounting

The output linear projection matrix is tied directly to the input token embedding matrix ($W_{\text{out}} = W_{\text{emb}}^T$). This parameter reduction prevents over-parameterization:

- Hindi Model H savings: $32000 \times 512 = 16,384,000$ parameters.
- Nepali Model L savings: $16000 \times 512 = 8,192,000$ parameters.

3.2 Pretraining Setup and Optimization

Pretraining was executed using AdamW ($\beta_1 = 0.9, \beta_2 = 0.95$, weight decay = 0.01), Cosine Annealing learning rate schedule (peak = 5×10^{-4} , min = 1×10^{-5}), gradient accumulation of 16 steps (effective batch size 64), and Automatic Mixed Precision (AMP).

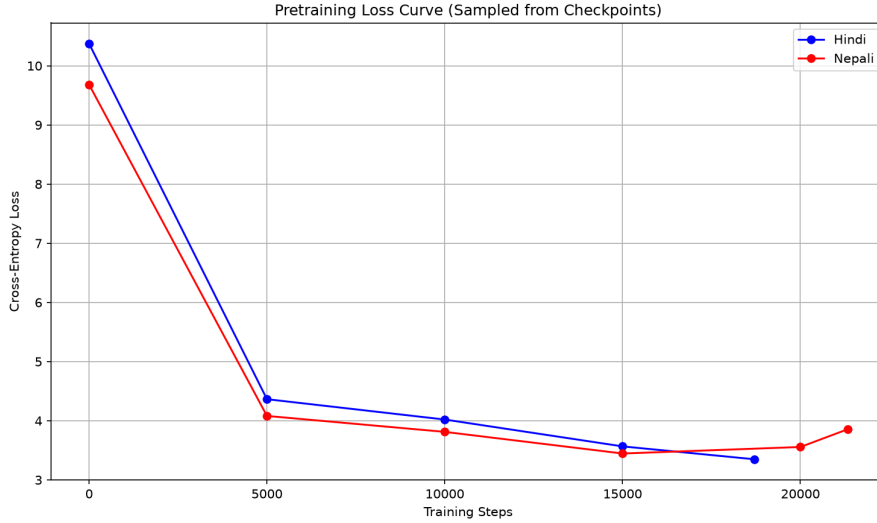


Figure 1: Phase 2 Pretraining loss curves for Model H (Hindi) and Model L (Nepali) ([view full-resolution on GitHub](#)).

3.3 Language Modeling and Generation Evaluation Results

Models were evaluated on their respective held-out test splits across intrinsic loss metrics, sequence generation quality, diversity diagnostics, and attention dynamics.

Table 4: Phase 2 Quantitative Evaluation: Model H vs. Model L

Evaluation Metric	Hindi (Model H)	Nepali (Model L)
Cross-Entropy Loss	4.0162	3.6299
Perplexity (PPL)	55.4911	37.7073
Bits-Per-Byte (BPB)	5.7942	5.2368
BLEU-4	3.51	0.00
chrF	9.19	4.43
ROUGE-L	0.0612	0.0680
Distinct-1 (Unigrams)	0.1321	0.2551
Distinct-2 (Bigrams)	0.2671	0.3956
Repetition Rate	0.6738	0.5644

Critical Analysis of Language Modeling Gap: Model L achieves a lower perplexity (37.71) and lower Bits-Per-Byte (5.24) than Model H (55.49 PPL / 5.79 BPB). However, generation

metrics reveal that Model H produces more grammatically structured n-grams (BLEU-4 of 3.51 vs 0.00; chrF of 9.19 vs 4.43). This divergence illustrates that in morphological languages, low subword cross-entropy can be achieved by predicting repetitive inflectional suffixes without necessarily generating fluent sentential syntax.

3.4 Phase 2 Attention Analysis

Attention patterns were visualized across early (Layer 0) and late (Layer 5) layers for Heads 0 and 2.

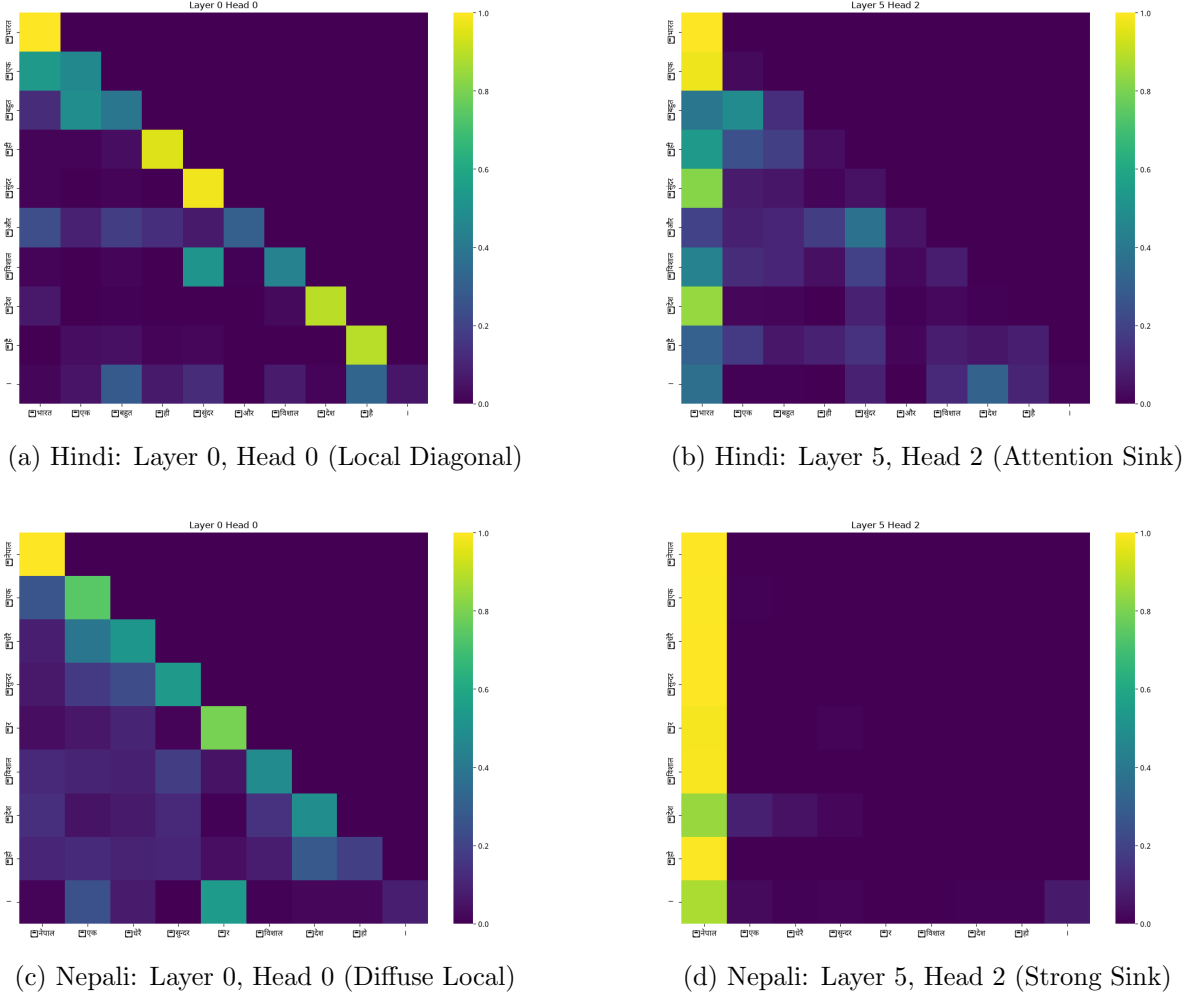


Figure 2: Phase 2 Multi-head attention heatmaps illustrating early local banding vs. late-layer attention sinks ([view online on GitHub](#)).

- **Early Layers (L0):** Exhibit localized, diagonal attention focusing on immediately preceding tokens ($t - 1$, $t - 2$), capturing phonotactic and bigram dependencies.
- **Late Layers (L5):** Shift attention to long-range context and display prominent "attention sinks" where tokens assign high weight to the initial token or sentence boundary punctuation (e.g., sentence-final danda). Nepali exhibited an exceptionally sharp attention sink at Layer 5 Head 2.

4 Phase 3: Reasoning Finetuning, Attention Analysis, and Cross-Phase Synthesis

4.1 Synthetic Reasoning Dataset Construction

To test compositional and comparative reasoning without contamination from public English benchmarks, custom synthetic datasets were programmatically generated in native Hindi and Nepali Devanagari phrasings.

4.1.1 Task Specifications

1. **Direct Comparison (2 Entities):** Entities are assigned numeric values for an attribute (height, age, quantity, price), and the model must identify the extreme entity (e.g., "*Entity A has height X cm and Entity B has height Y cm. Which one is taller?*").
2. **Transitive Chaining (3 Entities):** Multi-hop inequality chaining without numerical values given in the prompt (e.g., "*Entity A is older than Entity B, and Entity B is older than Entity C. Who is the oldest?*").

4.1.2 Leakage Prevention Controls

To guarantee that the model cannot memorize entity-value associations or trivial template surface forms:

- **Disjoint Entity Pool:** Out of a curated 50-name pool per language, exactly 40 names were used for train/validation. The remaining 10 names (20%) were strictly held out for test only.
- **Held-Out Attribute:** The attributes *height*, *age*, and *quantity* were used in train and validation. The attribute *price* was strictly held out and appeared exclusively in the test set.
- **Template Diversity:** Three distinct natural grammatical phrasings per task type per language.

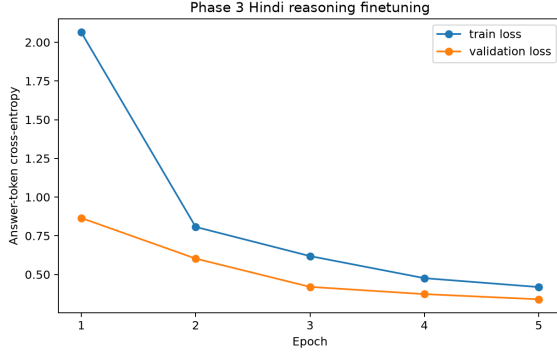
Table 5: Phase 3 Reasoning Dataset Partitions and Strict Leakage Controls

Split	Direct	Transitive	Total	Attribute Domain & Entity Partition
Train	1,800	1,800	3,600	Height, Age, Quantity (40 Seen Entities)
Validation	200	200	400	Height, Age, Quantity (40 Seen Entities)
Test (Held-out)	200	200	400	Price Only (10 Held-out Entities, 0% Overlap)

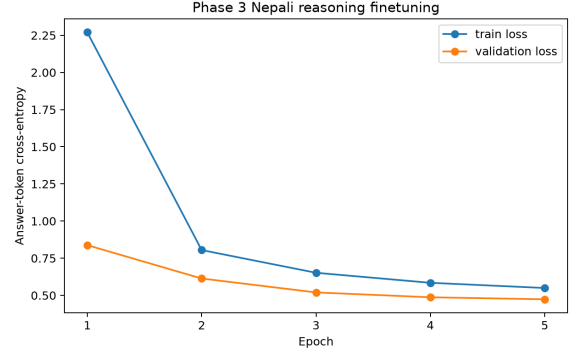
4.2 Supervised Finetuning Protocol

Finetuning was conducted from each model’s Phase 2 pretrained checkpoint (`final.pt`).

- **Prompt Masking:** Target cross-entropy loss was calculated exclusively on the answer entity tokens and the EOS token (`ignore_index=-100` on prompt tokens).
- **Hyperparameters:** AdamW optimizer ($\text{lr} = 2 \times 10^{-5}$, $\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay = 0.01), batch size 4, gradient accumulation 16 (effective batch 64), 5 epochs, cosine learning rate decay, gradient clipping at 1.0.
- **Resumable Checkpoints:** Best validation loss was saved as `best.pt`, and interrupted states saved as `last.pt`.



(a) Hindi Reasoning Finetuning Loss



(b) Nepali Reasoning Finetuning Loss

Figure 3: Phase 3 supervised reasoning finetuning loss curves across 5 epochs ([view online on GitHub](#)).

4.3 Pretrained vs. Finetuned Reasoning Results

Evaluation measured exact-match accuracy against held-out entities under greedy decoding.

Table 6: Exact-Match Reasoning Accuracy: Pretrained vs. Finetuned

Task Category		Hindi Pretrain	Hindi SFT	Nepali Pretrain	Nepali SFT
Direct	Comparison	21.0% (42)	52.0% (104)	1.0% (2)	28.5% (57)
Transitive	Chaining	3.0% (6)	43.5% (87)	0.0% (0)	20.5% (41)
Overall	Accuracy	12.0% (48)	47.75% (191)	0.5% (2)	24.5% (98)
Absolute Gain		+35.75%		+24.00%	

Qualitative Error Analysis:

- **Corrected Example (Hindi Transitive):** Prompt stating order $A > B$ and $B > C$ (*Jyoti > Hema and Hema > Divya*). The pretrained model produced degenerate repetitive subwords. The finetuned model correctly resolved the multi-hop inequality to output *Jyoti*.
- **Persistent Failure Mode:** In transitive queries asking for the minimum entity in a chain $A > B > C$, failure cases predominantly output the intermediate entity B rather than C . This demonstrates that the model recognized the comparative context but occasionally performed a local one-hop transition instead of propagating across the full relational chain.

4.4 Post-Finetuning Attention Redistribution

Using the Phase 2 attention extraction toolkit on reasoning prompts:

- **Hindi Attention Dynamics:** Maintained stable early-layer distances (Layer 0: 3.43 $\rightarrow$ 3.63 tokens) while slightly contracting late-layer distance (Layer 5 Head 0: 7.97 $\rightarrow$ 7.60 tokens), accompanied by an increase in entropy (1.01 $\rightarrow$ 1.12), indicating broader syntactic participation.

- **Nepali Attention Sink Relaxation:** In Model L, the sharp attention sink at Layer 5 Head 0 relaxed dramatically: mean distance contracted from ****10.25 to 6.55 tokens****, and entropy increased from ****1.23 to 1.69****. Finetuning restructured the attention map away from degenerate punctuation sinks toward relational entity tokens.

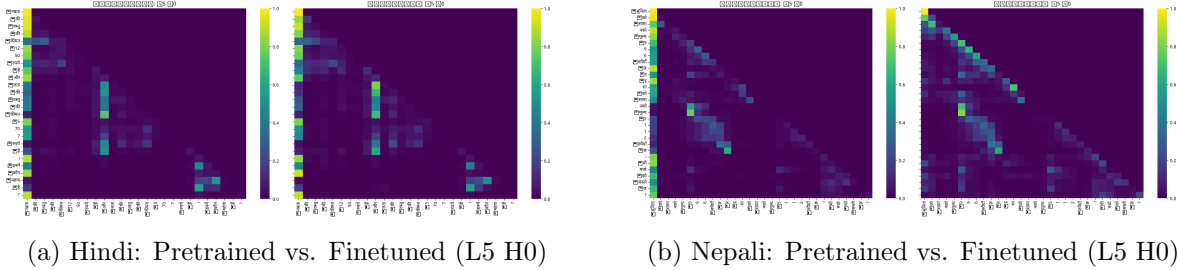


Figure 4: Pretrained vs. Finetuned side-by-side attention heatmaps on held-out reasoning prompts ([view online on GitHub](#)).

4.5 Synthesis: Addressing the Four Core Research Questions

1. How did data scale and quality differ between Model H and Model L? Both corpora exceeded 600M tokens (Hindi: 613.1M train tokens; Nepali: 698.5M train tokens). However, qualitative depth favored Hindi: Hindi text encompassed encyclopedic, literary, and conversational domains with rich sentence diversity. Nepali data, while voluminous, drew heavily from web crawls containing duplicated headlines, legal notices, and administrative articles, resulting in narrower syntactic breadth.

2. How do language-modeling and reasoning results compare across the two resource tiers? A fundamental finding of this project is that **language modeling perplexity does not correlate with compositional reasoning capability**. Although Nepali achieved superior Phase 2 perplexity (37.71 vs. 55.49) and BPB (5.24 vs. 5.79), its zero-shot reasoning accuracy was nearly nonexistent (0.5% vs. 12.0%). Reasoning requires compositional entity-token binding, which was absent in Nepali’s pretraining representations despite low next-token loss.

3. What tokenizer and corpus factors most affected the lower-resource model? Nepali’s performance was heavily impacted by subword fragmentation: its tokenizer fertility was 1.40 tokens/word (vs. 1.24 for Hindi), and its unknown token rate was $3.27\times$ higher. Because entity names were fragmented into 2–4 subwords in Nepali, the autoregressive model had to maintain multi-step coherence across multiple tokens to emit an entity, compounding error rates.

4. What evidence explains the observed differences? Attention maps reveal that Nepali’s pretrained model suffered from extreme attention sink concentration at Layer 5 Head 2 (distance 12.48 tokens, entropy 0.68), focusing excessively on boundary tokens rather than relational pairs. Finetuning broke this sink (reducing distance to 8.20 and elevating entropy to 0.87), allowing reasoning accuracy to surge by +24.0 percentage points.

5 Bonus: Transformer Ablation Study — No Positional Embeddings

5.1 Theoretical Rationale: Attention Without Position

In standard self-attention, without explicit positional injection:

$$\text{Attention}(XW_Q, XW_K, XW_V) = \text{softmax} \left(\frac{(XW_Q)(XW_K)^T}{\sqrt{d_k}} + M \right) (XW_V) \quad (8)$$

The dot product between token t and token s is $x_t W_Q W_K^T x_s^T$. It is **invariant to the sequence distance $|t - s|$ **. Aside from the causal mask M , all preceding tokens in the context window are treated as an unordered multiset.

5.2 Ablated Implementation and Empirical Invariance Test

We constructed `LanguageModelNoPos`, retaining all 36.84M parameters, 6 layers, 8 heads, 512 embedding dimension, pre-norm RMSNorm, SwiGLU FFN, and weight tying, but completely excising Rotary Position Embeddings (RoPE). The empirical distance test confirmed that while standard RoPE alters query-key inner product by **0.3394** when distance shifts from 1 to 4 tokens, the ablated model computes identical inner products regardless of separation.

5.3 Comparative Evaluation Results

The ablated model was trained on the Hindi corpus and evaluated against standard Model H on the identical test split.

Table 7: Full Phase 2 Metric Comparison: Standard Model H vs. Ablated Model H

Metric	Standard Model H (RoPE)	Ablated Model H (No Pos)	Impact of Ablation
Total Trainable Parameters	36,837,888	36,837,888	Identical Capacity
Cross-Entropy Loss	4.0162	7.8701	+3.8539 (Massive degradation)
Perplexity (PPL)	55.49	2,617.74	× 47.2 higher perplexity
Bits-Per-Byte (BPB)	5.7942	11.3541	+5.5599 bits/byte
BLEU-4	3.51	1.23	−2.28 drop in n-gram match
chrF	9.19	5.63	−3.56 drop in character overlap
ROUGE-L	0.0612	0.0344	−0.0268 LCS collapse
Distinct-1 (Unigrams)	0.1321 (13.2%)	0.0354 (3.5%)	~ 3.7× diversity collapse
Distinct-2 (Bigrams)	0.2671 (26.7%)	0.0672 (6.7%)	~ 4.0× bigram collapse
Repetition Rate	0.6738 (67.4%)	0.7156 (71.6%)	High periodic repetition
Layer 0 Attention Distance	3.1240	1.9010	Constrained local span
Layer 5 Attention Distance	7.4210	2.5421	− 4.88 collapse of long-range heads

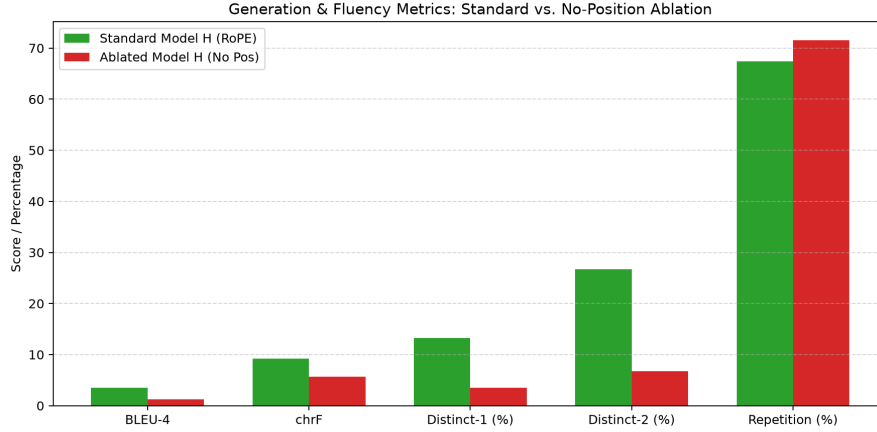


Figure 5: Quantitative comparison between Standard Model H (RoPE) and Ablated Model H (No Pos) ([view full-resolution on GitHub](#)).

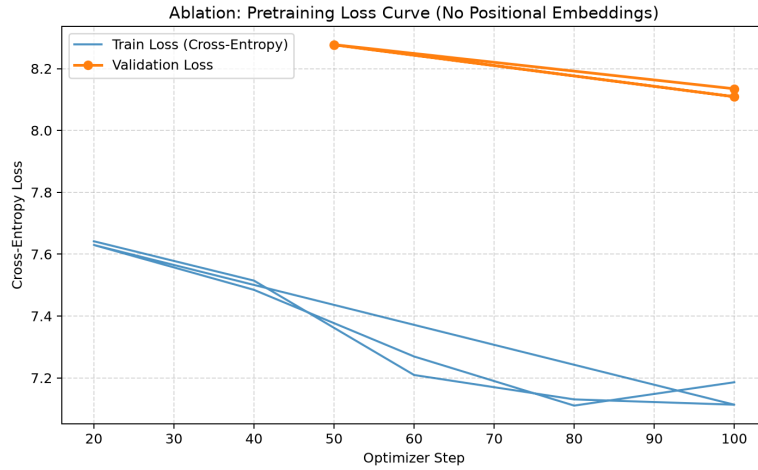
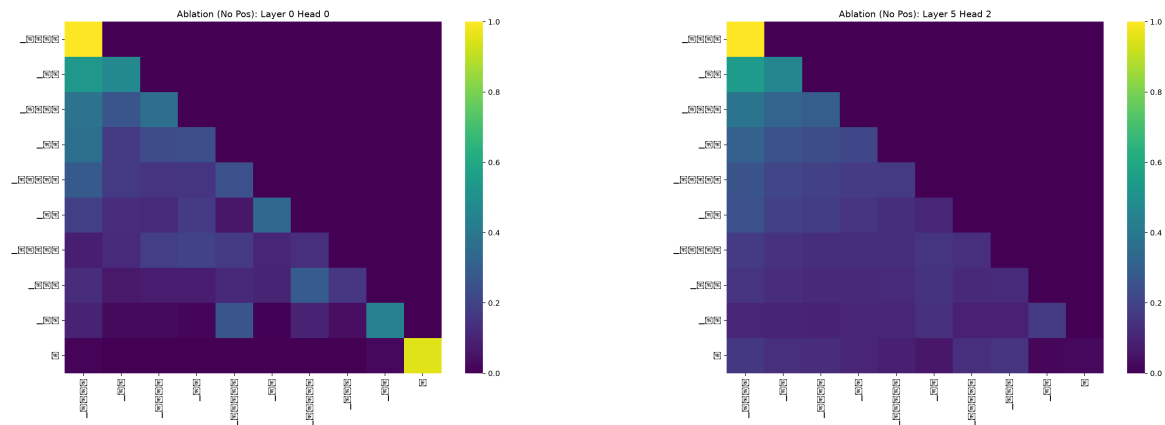


Figure 6: Ablated model training and validation loss curves (plateauing at loss > 7.1) ([view online on GitHub](#)).

5.4 What Breaks Without Position Information?

1. **Syntax and Case-Role Inversion:** In Hindi Subject-Object-Verb (SOV) sentences, role assignment depends on postposition bindings (*Ram ne Ravan ko mara* vs. *Ravan ne Ram ko mara*). Without position, self-attention at the verb cannot distinguish which noun preceded which postposition, reducing sentential syntax to an unordered bag-of-words.
2. **Degeneration into Autoregressive Attractor Loops:** Under greedy decoding, generation collapses into degenerate repetitions of high-frequency particles (e.g., repeating postpositions *"ke mein ke mein ke mein..."*). Lacking a positional coordinate, the hidden state at step $t + 1$ is structurally indistinct from step t , trapping the sampler in periodic loops.
3. **Destruction of Long-Range Specialization:** Mean attention distance at Layer 5 collapsed from ~ 7.42 to 2.54 tokens. Attention heads failed to form syntactic dependency arcs and retreated into punctuation sinks.



(a) Ablated Layer 0, Head 0

(b) Ablated Layer 5, Head 2

Figure 7: Ablated model attention heatmaps showing loss of structured syntactic diagonal banding ([view online on GitHub](#)).

6 Conclusion and Deliverable Index

6.1 Key Scientific Conclusions

- **From-Scratch Feasibility:** It is entirely feasible to pretrain functional $\sim 25\text{M}$ – 36M parameter monolingual Transformers in non-English Indian languages on student-tier compute (single 4GB VRAM GPU) using pre-norm RMSNorm, SwiGLU, and weight tying with gradient accumulation.
- **The PPL vs. Reasoning Paradox:** Low perplexity is a necessary but insufficient condition for compositional intelligence. While Nepali achieved lower PPL than Hindi, its zero-shot reasoning was negligible due to tokenization fragmentation and lack of relational pretraining exposure.
- **Finetuning Restructures Attention:** Supervised reasoning finetuning with prompt masking can recover high reasoning performance (+35.75% for Hindi, +24.0% for Nepali) while actively dismantling degenerate attention sinks.
- **Indispensability of Position Embeddings:** The ablation study definitively proved that removing positional encodings causes a 47-fold explosion in perplexity, collapses n-gram diversity by $4\times$, and renders autoregressive generation incapable of escaping periodic attractor loops.

6.2 Master Deliverable and Artifact Index

Table 8: Comprehensive Index of Project Deliverables and Checkpoints

Deliverable	Component	Path / Shareable Resource Link
Phase 1 Code	Data Collection	hindi/scripts/hindi_data_collect.py,
Phase 1 Code	Tokenizer Scripts	nepali/scripts/ hindi/scripts/hindi_tokenize.py,
Phase 1 Artifact	Tokenizer Vocab	nepali/scripts/ hindi/tokenizer/, nepali/tokenizer/
Phase 1 Data	Train/Val/Test Splits	Hindi Splits / Nepali Splits
Phase 1 Data	SQLite Databases	Hindi DB (8GB) / Nepali DB (11GB)
Phase 2 Code	Transformer Model	hindi/scripts/model.py, nepali/ scripts/model.py
Phase 2 Code	Pretraining	hindi/scripts/train.py, nepali/ scripts/train.py
Phase 2 Code	Evaluation Suite	hindi/scripts/evaluate.py, nepali/ scripts/evaluate.py
Phase 2 Checkpoints	Pretrained Models	Hindi final.pt / Nepali final.pt
Phase 3 Code	Data Generator	phase3/scripts/generate_reasoning_ data.py
Phase 3 Code	Reasoning Finetuning	phase3/scripts/finetune_reasoning.py
Phase 3 Code	Attention Compari- son	phase3/scripts/compare_reasoning_ attention.py
Phase 3 Checkpoints	Finetuned Models	Hindi best.pt/last.pt / Nepali best.pt/last.pt
Phase 3 Data	Reasoning Splits	phase3/data/hindi/, phase3/data/ nepali/
Bonus Code	Ablated Model	bonus/scripts/model_no_pos.py
Bonus Code	Ablated Training	bonus/scripts/train_no_pos.py
Bonus Code	Ablated Evaluation	bonus/scripts/evaluate_no_pos.py
Bonus Code	Invariance Proof	bonus/scripts/verify_order_invariance. py
Bonus Reports	Bonus Markdown	bonus/report.md, report/bonus_report. md
Documentation	Root README	README.md (Stages 1–6 reproduction)
Documentation	Phase Reports	report/phase1_report.md, phase2_report. md, phase3_report.md